

CU8B-I : A Unique 8-bit CPU Architecture with Reduced Fetch Cycle and Upward Compatibility with 8080/8085

Snehil Sanyal
Department of ECE
National Institute of Technology
Raipur, India
ssanyal.phd2024.ece@nitrr.ac.in

Shrish Verma
Department of ECE
National Institute of Technology
Raipur, India
shrishverma@nitrr.ac.in

Arman Sahu
Department of ECE
National Institute of Technology
Raipur, India
asahu014.btech2023@ece.nitrr.ac.in

Hetram
Department of ECE
National Institute of Technology
Raipur, India
hetram3234@gmail.com

Niti Jain
Department of ECE
National Institute of Technology
Raipur, India
njain067.btech2023@ece.nitrr.ac.in

Pradhyumn Dwivedi
Department of ECE
National Institute of Technology
Raipur, India
pdwivedi071.btech2023@ece.nitrr.ac.in

Abstract—This paper introduces CU8B-I, a novel 8-bit CPU architecture engineered for instruction fetch-cycle efficiency and upward compatibility with Intel’s 8080 and 8085 microprocessors. CU8B-I integrates handshake-based I/O, encoder-decoder subsystems, synchronous memory interface, and microcoded control. Its variable condition-aware instruction cycle mechanism gives it a unique architectural design. The code can be found at <https://github.com/pradhyumndwivedi/CU8B1>.

Index Terms—Custom Architecture, Fetch Cycle, Microcoded Control Unit, FPGA, Variable Machine Cycle, 8080, 8085

I. INTRODUCTION

Understanding the inner workings of microprocessors has long been based on the creation of experimental and instructional CPU architectures. Key architectural elements such as program counter, memory address register, accumulator, and basic control sequencing have become easier to understand due to the SAP-I (Fig.1) and SAP-II (Fig.2) models, which were first popularized by Malvino [1] and others [2]. While SAP-II expands the model with additional registers, bidirectional data buses, and micro-coded control ROM enabling support for a larger instruction set and more complex I/O mechanisms. SAP-I concentrates on instruction fetch-execute cycles with minimal hardware [3]. With its richer instruction set (approximately 246 instructions), interrupt handling, serial I/O, and effective clocking mechanisms, the Intel 8085 microprocessor marks a significant advancement toward commercial-grade architectures. In the late 1970s and early 1980s, it became a key component of embedded systems using a hardwired control unit and supporting peripheral-mapped and memory-mapped I/O [4]. By presenting architectures that combine simplicity and extensibility, recent academic efforts have attempted to close the gap between highly abstract educational CPUs and intricate commercial designs. A number of FPGA-

based microcoded processors have been created to support programmable I/O, increased instruction diversity, and the integration of real-world peripherals, all of which have been shown to improve research and teaching outcomes [5] [6]. In contrast to SAP-I/II, CU8B-I allows for dynamic T-state

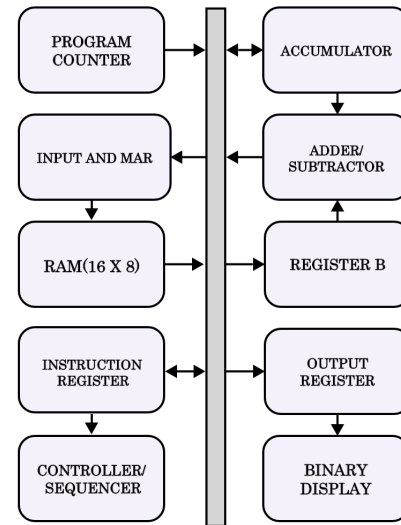


Fig. 1. Simple as Possible (SAP-I) Architecture

optimization by supporting variable-length machine cycles, especially via mechanisms like condition-aware instruction cycling and NOP detection. It maintains the compatibility of the instruction set with 8085/8080. CU8B-I differentiates itself from other traditional 8-bit CPUs by integrating a 32-bit microinstruction format to facilitate modular control logic and

branching optimization. This enables instruction expansion and vertical microprogramming without requiring an architectural change. The register file (which includes TEMP and

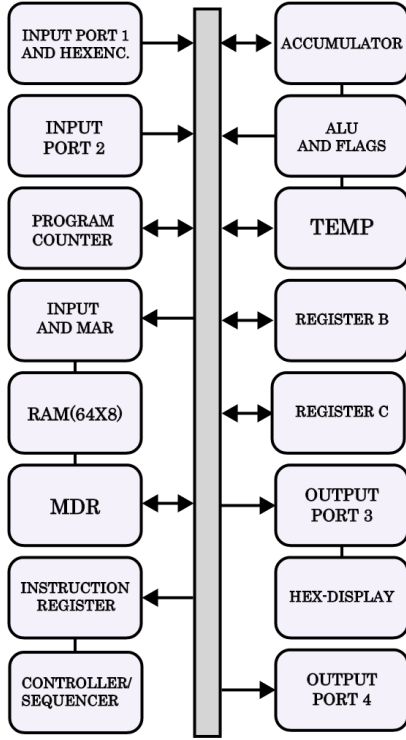


Fig. 2. Simple as Possible (SAP-II) Architecture

WZ registers) and encoder-decoder subsystems offer more flexibility to users. CU8B-I supports ongoing initiatives in the open-source and academic communities to create scalable, reusable microprocessor cores that support realistic computational models and maintain academic transparency.

II. ARCHITECTURE

A. Arithmetic Logic Unit (ALU)

A wide variety of twelve operations are supported by the Arithmetic Logic Unit (ALU) in CU8B-I. These consist of logical operations like AND, OR, XOR, and NOT in addition to common arithmetic operations like addition and subtraction. Additionally, it manages more complex functions like comparison instructions, addition with carry, left and right rotation, and bit shifts.

B. Register Bank

Eight 8-bit general-purpose registers with the designations A, B, C, D, E, H, L, and TEMP make up the Register Bank of CU8B-I. These provide dual-purpose access for both 8-bit and 16-bit operations and are paired into three register pairs: BC, DE, and HL. Special purpose registers, such as W and Z, are used for tasks like swapping, temporary storage, or accessing indirect memory operations. The TEMP register serves as an

intermediary holding location. Four status flags—Parity (P), Zero (Z), Sign (S), and Carry (C)—are stored in a specific Flag Register and are updated in response to ALU operations. For conditional instructions and control flow decisions, such as branches and loops, these flags are crucial.

C. Memory

The size of the Memory is 64 KB. CU8B-I employs a dual-register structure for memory communication MAR (Memory Address Register) holds the effective address, and MDR (Memory Data Register) stores data for read/write cycles. Use of MAR and MDR enables synchronous memory access.

D. I/O Ports

A 16-bit input port, Port 1, is used to receive external data in parallel via the hexadecimal encoder. Protocols requiring device synchronization and acknowledgement use Port 2, a handshake-driven 8-bit input port. Real-time data transmission to external devices is possible using Port 3, which is a 16-bit output port. This port also serves as a hexadecimal decoder, transforming binary-coded inputs into outputs that are compatible with hex displays. The READY-ACK handshake interface ensures synchronized data exchange between the processor and peripheral devices, and it takes place via 8-bit output Port 4.

E. Control Unit

Its 32-bit Microcoded Control Unit, which decodes each instruction into a set of control words, is at the heart of the system. It has a control ROM of 4 KB. It also has a presettable counter to adapt according to the conditional instruction and terminate it with a NOP operation.

III. INSTRUCTION SET ARCHITECTURE (ISA)

The CU8B-I instruction set contains over 200 extendable instructions. The instructions are Data Transfer Instructions (e.g., MOV, STA, LDA), Arithmetic and Logical Operations (e.g., ADD, SUB, AND, OR, XOR), and Program Control (e.g., JMP, CALL, RET). Instruction lengths vary between one and three bytes depending on addressing mode (immediate, direct, indirect, register, or stack-based).

IV. EXPERIMENT AND IMPLEMENTATION

A. Experiment

The CU8B-I processor was synthesized and validated using Xilinx Vivado Design Suite 2024.2 and implemented on a Xilinx XC7S50CSGA324-1 FPGA. The design cycle included modeling, synthesis, implementation, simulation, and performance benchmarking. The actions listed below were taken

- **Design and Simulation:** Verilog HDL [7] was used to describe modules like the ALU, register file, control unit, memory interface, and I/O ports. Testbenches were created to verify that control signal sequencing, I/O timing, and core instructions (such as MOV, ADD, JMP, IN, and OUT) were executed correctly.

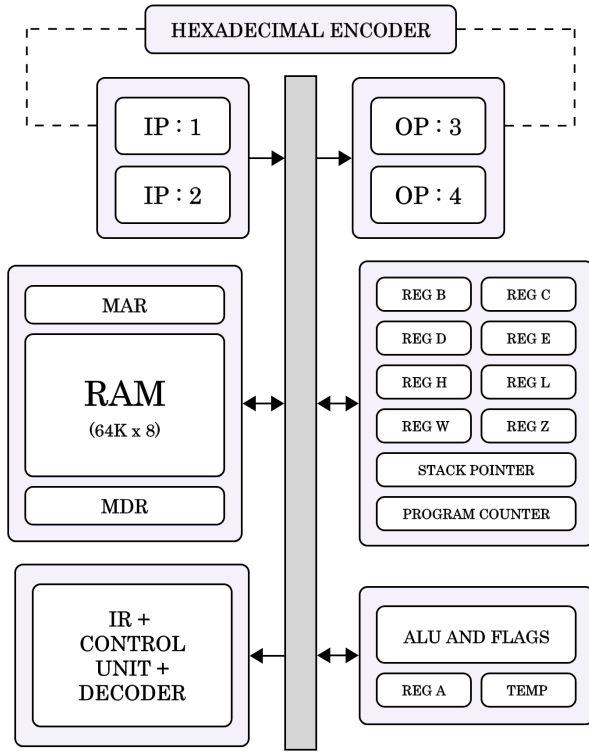


Fig. 3. CU8B-I Architecture

- **Functional Testing:** A series of benchmark programs were executed to evaluate memory interactions, conditional jumps, I/O responsiveness, and ALU accuracy.
- **Power and Thermal Analysis:** To check the power consumption and thermal behavior, Vivado Power Estimator (XPE) and Xilinx XPower Analyzer were used. A peak junction temperature of 78°C was observed from a 10-minute continuous simulation.

B. Performance Evaluation

TABLE I
PERFORMANCE MATRIX OF CU8B-I ARCHITECTURE

Metric	Measured Value
Logic Utilization (LUTs)	14,085 (99.9%)
Flip-Flop Utilization	33,231
Dynamic Power Consumption	27.7 W
I/O Power Consumption	14.7 W
Clock Frequency (F_{max})	~93 MHz
Instruction Cycle Time	2–3 T-states (avg.)
Microcoded ROM Size	32-bit × 256 instructions
Thermal Profile (Peak Temp.)	~78°C
I/O Throughput	Up to 64 Mbps
Total On-Chip RAM Usage	19%

The unique control structure and I/O subsystems of the CU8B-I enable its dense logic utilization. The pipelining and latching of micro-instruction states is reflected in the high flip-flop usage. Internal signal switching, especially in control and routing logic, increases dynamic power consumption. Similarly, the parallel handshaking mechanisms make I/O power

important. Despite reaching a clock frequency of roughly 93 MHz, the design is just constrained by memory access latency and routing depth. However, compared to legacy counterparts, the shortened fetch-execute cycle (2–3 T-states) provides a faster instruction throughput. The control flow is flexible and scalable due to the microcoded ROM structure. The thermal profile suggests that the effective memory use guarantees the proper operation of control ROM, buffer registers, and MAR/MDR modules and will be implemented in future versions.

C. Key Experimental Outcomes

During testing, the architecture showed a number of significant results. Faster execution and fewer average cycles per instruction were made possible by optimized control sequencing. Strong microcode flexibility was demonstrated by the system; new instructions could be added without affecting the datapath by simply updating the contents of the ROM. Handshake-controlled parallel ports were used to achieve deterministic I/O communication, providing a dependable real-time peripheral interface.

V. COMPARISON WITH RELATED ARCHITECTURES

A. Comparison with SAP-I/II/III & 8085

The CU8B-I processor offers more flexibility and a smarter design compared to older 8-bit systems such as the SAP-I/II/III series and Intel's 8085. Early SAP processors were built mainly for teaching and could perform only very basic tasks such as simple data transfer and arithmetic. They had only 8–16 instructions, worked on fixed control logic, and could not adapt to complex or real-time operations. In comparison, CU8B-I introduces a microcoded control unit that allows it to modify or add new features without changing hardware, making it much more suitable for both research and practical embedded use. The Intel 8085 processor was a big improvement over SAP architectures, adding features such as interrupts, a multiplexed address/data bus, and a large instruction set. However, the 8085 still used a hardwired control unit and relied on a fixed number of cycles per instruction. These static cycles cause lower performance because even simple operations take the same time as complex ones. In addition, the 8085's I/O operations depended largely on software-controlled handshaking, which increased processing delay. CU8B-I solves these problems with several key innovations. Its microcoded controller separates the tasks of incrementing the program counter and loading the memory address register, allowing instructions to be fetched in just 2 T-states, almost twice as fast as the 8085. The 32-bit microcoded control word precisely controls all data paths properly inside the processor, minimizing control overhead and enabling easy instruction set expansion. CU8B-I has a variable machine cycle mechanism that automatically adjusts the execution time depending on how complex an instruction is. It uses an NOP-triggered preset counter to recognize the end of an instruction, making simpler instructions complete more quickly, while complex ones

TABLE II
ARCHITECTURE COMPARISON TABLE

Feature	CU8B-I	CMOS(8-bit)	Micro-FIMEE-08	8-bit μ P	FPGA 8-bit	Abu-Reihan
Control Unit	32-bit microcoded control ROM	Clocked CMOS hardwired logic	Simple sequential control	Wilkes-style microprogram	Finite State Machine	Hardwired logic
Fetch Cycle	2 T-states	4 T-states	4 T-states	Sequential	4-6 T-states	4 T-states
Bus Width	16-bit	8-bit	8-bit data / 16-bit address	8-bit	8-bit	8-bit
Instruction Set	213 extendable instructions	8080A-compatible fixed set	100 instructions	42 instructions	30 instructions	Limited educational ISA
Branch Logic	CJR branch prediction	-	Conditional branching	Basic conditional flags	-	-
IO System	READY-ACK handshake IO	Simple input/output	Basic IO ports	Memory-mapped IO	Register-based IO	Instruction-based IO
Memory System	Dual-register MAR-MDR	Single-bus memory	Single Von Neumann memory	Single-bus	Simple SRAM (16x8)	Single memory path

take only as long as needed. This adaptive timing improves the performance and efficiency, which is not possible in SAP or 8085 architectures. CU8B-I also introduces the Conditional Jump Rejected (CJR) signal, which is a simple form of conditional branch prediction. This feature allows the processor to evaluate and decide on conditional jumps within the control unit itself without using additional external instructions. This helps to reduce delays and make branching faster and smoother compared to fixed-logic processors like SAP and 8085. In terms of I/O performance, CU8B-I uses hardware-based handshaking through READY-ACK signals through dedicated ports (IP1-IP2 and OP3-OP4). This ensures synchronized and low-latency data transfers, improving communication speed with sensors and peripherals, unlike older architectures that depended on software-driven “OUT” instructions.

However, because the 8085 only uses IN and OUT instructions instead of physical handshake logic, it is more dependent on polling or software-level verification. The CU8B-I’s memory architecture makes use of a dual-register structure that operates in synchronous mode with MAR and MDR. This configuration gives more control over read/write timing and makes testbench-driven verification with FPGA simulation easier. CU8B-I provides a more reliable memory interface when compared to SAP-I’s straightforward memory model and 8085’s multiplexed memory access.

B. Comparison with Other Architectures

When compared to the CMOS 8-bit microprocessor (Toshiba, 1981) [8], CU8B-I achieves a huge increase in operating frequency from 3 MHz to 93 MHz while retaining 8080/8085 compatibility. The CMOS prototype relied on fixed-timing instruction cycles and basic IO interfacing. In relation to educational 8-bit processors such as Micro-FIMEE-08 [9] and the Model of 8-bit Microprocessor (Wilkes-inspired design) [10], CU8B-I transitions from an academic to a research-capable platform. Micro-FIMEE-08, while open-source and suitable for teaching, uses a static control structure and limited instruction expansion. In contrast, CU8B-I inte-

grates branch prediction (CJR logic), dual-fetch sequencing, and dynamic T-state optimization, achieving higher computational throughput while maintaining flexibility for academic use. The CU8B-I also surpasses basic FPGA-based implementations such as FPGA Implementation of an 8-bit Simple Processor [11] and Abu-Reihan CPU (Sharif University) [12]. These earlier designs focused on academic clarity or minimal FPGA utilization but lacked advanced control mechanisms, timing optimization, and compatibility with modern tools. CU8B-I, validated on a Xilinx XC7S50 FPGA using Vivado 2024.2, reached 93 MHz with complete thermal and power profiling, confirming its capability for real-time embedded applications. Overall, the CU8B-I architecture combines performance efficiency, design modularity, and educational accessibility.

VI. CONCLUSION AND PROSPECT

The CU8B-I architecture improves traditional 8-bit processor designs by combining microcoded intelligent control and optimized instruction fetching. It achieves nearly 50% higher fetch efficiency compared to the 8085 while maintaining compatibility with classic architectures. The system’s FPGA-based implementation confirms that CU8B-I is a practical and academically feasible design.

Through its dual-register memory system, branch prediction logic, and handshake I/O mechanism, CU8B-I offers a balance between speed, flexibility, and reliability. It can serve educational and embedded system purposes, helping students understand microprocessor design while providing a scalable foundation for real-world applications.

Future work includes:

- Adding an interrupt controller to improve the real-time response.
- Enhancing power and thermal management for stable operation under continuous loads.
- Expanding the instruction set to support application-specific tasks.
- Integrating block RAM and DSP cores for AIoT and signal processing applications.

Overall, CU8B-I preserves the simplicity of early micro-computers while adopting modern design features that make it suitable for research, education, and embedded innovation.

ACKNOWLEDGMENT

The authors acknowledge the faculty and staff of the Department of ECE, NIT Raipur, who helped with this project. We would also like to thank the Vivado FPGA lab team for helping with the testing and synthesis.

REFERENCES

- [1] A. P. Malvino, *Digital Computer Electronics*. Tata McGraw-Hill, 1993.
- [2] M. Mano and C. R. Kime, *Logic and Computer Design Fundamentals*. Pearson, 4th ed., 2007.
- [3] M. Design, "Vivado design suite user guide," 2012.
- [4] S. Brown and Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*. McGraw-Hill, 2013.
- [5] J. L. Hennessy and D. A. Patterson, *Computer architecture: a quantitative approach*. Elsevier, 2011.
- [6] N. H. Weste and D. Harris, *CMOS VLSI design: a circuits and systems perspective*. Pearson Education India, 2015.
- [7] "Ieee standard for verilog hardware description language," *IEEE Std 1364-2005 (Revision of IEEE Std 1364-2001)*, pp. 1–590, 2006.
- [8] Y. Suzuki, Y. Saeki, M. Takata, Y. Sato, M. Kunieda, and S. Takizawa, "C/sup 2/mos 8-bit microprocessor," *IEEE Journal of Solid-State Circuits*, vol. 16, no. 5, pp. 599–601, 1981.
- [9] R.-T. de J, A. Ordaz-Moreno, J. A. Vite-frias, and A. Garcia-Perez, "8-bit cisc microprocessor core for teaching applications in the digital systems laboratory," in *2006 IEEE International Conference on Reconfigurable Computing and FPGA's (ReConFig 2006)*, pp. 1–5, 2006.
- [10] J. Divić and M. Debeljuh, "Model of 8-bit microprocessor intended for lecturing," in *2012 Proceedings of the 35th International Convention MIPRO*, pp. 1320–1324, 2012.
- [11] E. Ayeh, K. Agbedanu, Y. Morita, O. Adamo, and P. Guturu, "Fpga implementation of an 8-bit simple processor," in *2008 IEEE Region 5 Conference*, pp. 1–5, 2008.
- [12] M. Ajdari and M. Tabandeh, "Design and construction of an 8-bit computer, along with the design of its graphical simulator for pedagogical purposes," in *2012 15th International Conference on Interactive Collaborative Learning (ICL)*, pp. 1–5, 2012.